

1. K8s 기초 요약

쿠버네티스 핵심 구성 요소는 무엇이며 어떻게 활용되나요? 쿠버네티스는 **컨테이너 오케스트레이션**을 통해 애플리케이션의 고가용성, 확장성, 재해 복구를 보장하며, Pod, Service, Deployment 등 다양한 구성 요소를 활용하여 복잡한 애플리케이션을 효율적으로 관리합니다.

1. 쿠버네티스 소개 및 기본 개념

쿠버네티스는 복잡한 애플리케이션을 효율적으로 관리하기 위한 **컨테이너 오케스트레이션** 프레임워크이다.

1.1. 쿠버네티스란 무엇인가?

1. **쿠버네티스(Kubernetes)**는 구글에서 개발한 **오픈 소스 컨테이너 오케스트레이션 프레임워크**이다.
2. 도커 컨테이너를 포함한 다양한 컨테이너를 관리하며, 수백, 수천 개의 컨테이너로 구성된 애플리케이션을 다양한 환경(물리적 머신, 가상 머신, 클라우드, 하이브리드)에서 관리할 수 있게 한다.

1.2. 쿠버네티스의 필요성 및 해결 과제

1. **마이크로서비스 아키텍처**의 등장으로 컨테이너 사용이 급증했으며, 애플리케이션은 수백, 수천 개의 컨테이너로 구성되는 경우가 많아졌다.
2. 이러한 복잡한 환경에서 컨테이너를 스크립트나 자체 제작 도구로 관리하는 것은 매우 어렵거나 불가능하다.
3. 이에 따라 **컨테이너 오케스트레이션 기술**의 필요성이 대두되었다.
4. 쿠버네티스와 같은 오케스트레이션 도구는 다음과 같은 기능을 보장한다.
 - a. **고가용성**: 애플리케이션이 다운타임 없이 항상 사용자에게 접근 가능하도록 보장한다.
 - b. **확장성**: 애플리케이션 부하 증가 시 빠르게 확장하고, 부하 감소 시 쉽게 축소하여 유연하게 대응할 수 있다.
 - c. **재해 복구**: 데이터 손실, 서버 고장 등 인프라 문제 발생 시 데이터를 백업하고 복원할 수 있는 메커니즘을 제공한다.

1.3. 쿠버네티스 아키텍처 개요

1. 쿠버네티스 클러스터는 최소 하나 이상의 **마스터 노드**와 여러 개의 **워커 노드**로 구성된다.
2. **워커 노드**:
 - a. 실제 작업이 이루어지는 곳으로, 애플리케이션 컨테이너가 배포된다.
 - b. 각 워커 노드에는 **큐블릿(Kubelet)** 프로세스가 실행되며, 이는 쿠버네티스 프로세스로 클러스터 통신 및 애플리케이션 실행을 담당한다.
 - c. 워크로드 분산 방식에 따라 실행되는 컨테이너 수가 달라지며, 일반적으로 크기가 크고 많은 리소스를 보유한다.
3. **마스터 노드**:
 - a. 클러스터 실행 및 관리에 필수적인 여러 쿠버네티스 프로세스를 실행한다.
 - b. 주요 프로세스는 다음과 같다.
 - i. **API 서버**: 클러스터의 진입점 역할을 하며, 모든 클라이언트(UI, 스크립트, 명령줄 도구)가 통신하는 대상이다.
 - ii. **컨트롤러 관리자**: 클러스터 상황을 파악하고 복구가 필요한 경우를 관리한다.
 - iii. **스케줄러**: 워크로드와 노드 리소스를 기반으로 컨테이너를 여러 노드에 분산 배치한다.
 - c. **etcd 키-값 저장소**: 클러스터의 현재 상태, 구성 데이터, 노드 및 컨테이너 상태 데이터를 저장하며, 클러스터 상태 복구에 사용된다.
 - d. **가상 네트워크**: 모든 노드를 연결하여 클러스터 내 모든 노드를 하나의 강력한 머신처럼 작동하게 한다.
4. 마스터 노드는 워커 노드보다 리소스 요구량이 적지만, 클러스터 접근 권한을 잃을 수 있어 매우 중요하며 백업이 필수적이다.
5. 프로덕션 환경에서는 일반적으로 최소 두 개의 마스터 노드를 운영하여 고가용성을 확보한다.

2. 쿠버네티스의 핵심 구성 요소

쿠버네티스

는 **Pod, Service, Deployment, ConfigMap, Secret, Volume, StatefulSet** 등 다양한 구성 요소를 통해 애플리케이션을 효율적으로 관리한다.

2.1. Pod: 컨테이너의 추상화 계층

1. **Pod**는 쿠버네티스의 가장 작은 배포 단위이며, 컨테이너 위에 추상화된 계층이다.
2. 컨테이너 런타임 기술을 추상화하여 필요에 따라 교체할 수 있게 하고, 사용자는 Kubernetes 계층과만 상호 작용하면 된다.
3. 일반적으로 하나의 Pod는 하나의 애플리케이션 컨테이너를 실행하도록 설계되었지만, 메인 애플리케이션 컨테이너와 도우미 컨테이너(사이드카)를 함께 실행할 수도 있다.
4. 각 Pod는 컨테이너가 아닌 **고유한 IP 주소**를 할당받아 서로 통신할 수 있다.
5. Pod는 **일시적(ephemeral)**이라는 특성이 있어, 컨테이너 충돌, 서버 리소스 부족 등으로 인해 쉽게 소멸되고 새로운 Pod가 생성될 수 있다.

2.2. Service: 안정적인 네트워크 접근 제공

1. Pod는 일시적이어서 IP 주소가 변경될 수 있으므로, **Service**를 사용하여 안정적인 네트워크 접근을 제공한다.
2. Service는 각 Pod에 할당되는 **고정 IP 주소 또는 영구 IP 주소** 역할을 한다.
3. Pod가 소멸되어도 Service는 계속 실행되며 IP 주소가 유지되므로, 엔드포인트 변경 없이 Pod에 접근할 수 있다.
4. Service는 크게 두 가지 유형으로 나뉜다.
 - a. **외부 서비스**: 외부 소스와의 통신을 허용한다.
 - b. **내부 서비스**: 서비스 생성 시 지정된 내부에서만 접근 가능하다.
5. 외부 서비스의 URL은 일반적으로 노드 IP 주소와 서비스 포트 번호를 포함하며, 테스트 목적에는 적합하지만 최종 제품에는 보안 및 편의성을 위해 추가적인 구성이 필요하다.

2.3. Ingress: 외부 트래픽 라우팅

1. **Ingress**는 서비스 대신 요청을 먼저 받아 포워딩한 후 서비스로 전달하는 구성 요소이다.
2. 이를 통해 애플리케이션과 안전한 프로토콜 및 도메인 이름을 사용하여 통신할 수 있다.

2.4. ConfigMap 및 Secret: 외부 구성 관리

1. **ConfigMap**:
 - a. 애플리케이션의 **외부 구성 파일** 역할을 하며, 데이터베이스 URL과 같은 구성 데이터를 저장한다.

- b. ConfigMap을 Pod에 연결하여 데이터를 가져오도록 할 수 있으며, 구성 변경 시 ConfigMap만 수정하면 되므로 애플리케이션 재빌드가 필요 없다.

2. Secret:

- a. ConfigMap과 유사하지만, **자격 증명(비밀번호, 인증서 등)**과 같은 비밀 데이터를 저장하는 데 사용된다.
 - b. Secret은 평문이 아닌 **Base64로 인코딩된 형식**으로 저장되며, 보안 강화를 위해 추가적인 암호화가 권장된다.
3. ConfigMap과 Secret의 데이터는 애플리케이션 파드 내에서 환경 변수나 속성 파일 등을 통해 활용될 수 있다.

2.5. Volume: 데이터 영구 저장

1. **Volume**은 물리적인 하드 드라이브 스토리지를 Pod에 연결하는 역할을 한다.
2. 스토리지는 로컬 머신 또는 원격 스토리지(클라우드 스토리지 등)가 될 수 있다.
3. 이를 통해 데이터베이스 컨테이너나 Pod가 재시작되어도 데이터가 유지된다.
4. Kubernetes 클러스터는 데이터 영구 저장을 명시적으로 관리하지 않으며, 사용자 또는 관리자가 데이터 백업, 복제, 관리를 책임져야 한다.

2.6. Deployment: 애플리케이션 복제 및 관리

1. 애플리케이션 Pod가 다운되거나 재시작될 때 발생하는 다운타임을 방지하기 위해 **분산 시스템과 컨테이너 복제**가 사용된다.
2. **Deployment**는 Pod 복제본을 생성하고 관리하는 블루프린트 역할을 한다.
3. Deployment를 통해 Pod 복제본 수를 지정하고 필요에 따라 확장 또는 축소할 수 있다.
4. Pod가 컨테이너 위에 있는 추상화 계층이라면, Deployment는 Pod 위에 있는 또 다른 추상화 계층으로 Pod와의 상호 작용, 복제 등을 더 편리하게 해준다.
5. Service는 로드 밸런싱 역할도 수행하여, 요청을 받아 다른 노드로 전달한다.

2.7. StatefulSet: 상태 저장 애플리케이션 관리

1. 데이터베이스와 같이 **상태(데이터)**를 가지는 애플리케이션은 Deployment로 복제할 수 없으며, **StatefulSet**을 사용해야 한다.
2. StatefulSet은 MySQL, MongoDB, Elasticsearch 등 상태 저장 애플리케이션 또는 데이터베이스를 위해 특별히 설계되었다.

3. StatefulSet은 Deployment와 마찬가지로 Pod를 복제하고 확장/축소하지만, 데이터 베이스 읽기/쓰기를 동기화하여 데이터 불일치를 방지한다.
4. Kubernetes 클러스터 외부에 데이터베이스를 호스팅하고, 클러스터 내부에는 상태 비 저장 애플리케이션(Deployment 사용)만 두는 것이 일반적이다.

2.8. Kubernetes 구성 요소 요약

1. **Pod**: 컨테이너의 추상화 계층.
2. **Service**: Pod 간 통신 및 안정적인 네트워크 접근 제공.
3. **Ingress**: 클러스터로 트래픽 라우팅.
4. **ConfigMap**: 외부 구성 데이터 관리.
5. **Secret**: 민감한 정보(자격 증명 등) 관리.
6. **Volume**: 데이터 영구 저장.
7. **Deployment**: Pod 복제 메커니즘 (애플리케이션 청사진).
8. **StatefulSet**: 상태 저장 애플리케이션(데이터베이스 등) 특화 관리.

3. 쿠버네티스 환경 설정 및 배포

로컬 환경에서 쿠버네티스를 경험하기 위해 **Minikube**와 **kubectl**을 설치하고, **MongoDB** 데이터베이스와 **웹 애플리케이션**을 포함한 실제적인 애플리케이션을 배포하는 과정을 다룬다.

3.1. Minikube와 kubectl 소개 및 설치

1. **Minikube**:
 - a. 로컬 환경에서 쿠버네티스 클러스터를 쉽게 테스트하고 구축할 수 있게 해주는 **오픈 소스 도구**이다.
 - b. 마스터 프로세스와 워커 프로세스가 모두 **하나의 노드**에서 실행되는 단일 노드 클러스터이다.
 - c. Docker 컨테이너 런타임이 미리 설치되어 있어 컨테이너 또는 Pod 실행이 가능하다.
2. **kubectl**:
 - a. Kubernetes 클러스터와 상호 작용하기 위한 **명령줄 도구**이다.
 - b. API 서버에 명령을 제출하여 구성 요소 생성, 삭제 등의 작업을 수행한다.

- c. 클라우드 클러스터, 하이브리드 클러스터 등 어떤 유형의 Kubernetes 클러스터 설정에도 사용될 수 있다.

3. 설치:

- a. Minikube는 운영 체제 및 아키텍처에 따라 설치 방법이 다르며, 공식 문서를 참조하는 것이 좋다.
- b. Mac OS에서는 Homebrew를 사용하여 `brew install minikube` 명령으로 설치할 수 있다.
- c. Minikube를 실행하려면 **Docker와 같은 컨테이너 또는 가상 머신 도구**가 필요하며, Docker가 권장 드라이버이다.
- d. Docker Desktop은 공식 웹사이트에서 다운로드 및 설치할 수 있다.

4. Minikube 클러스터 시작:

- a. `minikube start` 명령어를 실행하여 클러스터를 시작한다.
- b. 처음 실행 시 클러스터 생성 및 이미지 다운로드에 시간이 소요될 수 있다.
- c. `minikube status` 명령으로 클러스터 상태를 확인할 수 있다.

5. kubectl 사용:

- a. Minikube 설치 시 종속성으로 함께 설치된다.
- b. `kubectl get nodes` 명령으로 클러스터의 모든 노드 정보를 확인할 수 있다.

3.2. 애플리케이션 배포를 위한 Kubernetes 구성 파일 생성

1. **구성 파일 구조:** Kubernetes 구성 파일은 일반적으로 세 부분으로 구성된다.
 - a. **메타데이터:** 생성하는 구성 요소의 정보(이름 등)를 저장한다.
 - b. **명세(Spec):** 해당 구성 요소에 적용하려는 모든 종류의 구성을 정의한다.
 - c. **상태(Status):** Kubernetes에서 자동으로 생성 및 편집되며, 원하는 상태와 실제 상태를 비교하여 자가 복구를 수행한다.
2. **YAML 형식:** Kubernetes 구성은 YAML 형식으로 작성되며, 들여쓰기에 엄격해야 한다.
3. **ConfigMap 생성 (`mongoconfig.yml`):**
 - a. MongoDB 서비스의 URL을 저장하는 ConfigMap을 생성한다.
 - b. `data` 속성에 키-값 쌍으로 구성 데이터를 정의한다.
4. **Secret 생성 (`mongo-secret.yml`):**

- a. MongoDB 사용자 이름과 비밀번호를 저장하는 Secret을 생성한다.
- b. Secret의 값은 Base64로 인코딩해야 한다.
- c. `data` 속성에 키-값 쌍으로 비밀 데이터를 정의한다.

5. MongoDB 배포 및 서비스 생성 (`mongo.yaml`):

a. Deployment:

- i. MongoDB 이미지(예: `mongo:5.0`)를 사용하는 Pod를 생성하도록 구성한다.
- ii. **Labels:** 구성 요소 식별을 위한 키-값 쌍으로, Pod와 Deployment 간의 연결에 사용된다.
- iii. **Selector** `matchLabels`: Deployment가 어떤 Pod에 속하는지 식별하는 기준이 된다.
- iv. **Replicas:** 생성할 Pod의 수를 지정하며, 데이터베이스는 일반적으로 1개로 설정한다.

b. Service:

- i. MongoDB에 접근할 수 있는 고정 IP 주소와 포트를 제공한다.
- ii. **Selector:** Deployment의 Labels와 일치하는 Pod로 요청을 전달한다.
- iii. **Port:** Service 자체의 포트.
- iv. **TargetPort:** Service가 요청을 전달할 Pod의 포트 (일반적으로 컨테이너 포트와 동일).

6. 웹 애플리케이션 배포 및 서비스 생성 (`webapp.yaml`):

a. Deployment:

- i. 웹 애플리케이션 이미지(예: Docker Hub의 공개 이미지)를 사용한다.
- ii. 컨테이너 포트는 3000번으로 설정한다.
- iii. **Replicas:** 애플리케이션의 고가용성을 위해 여러 개로 설정할 수 있다.

b. Service:

- i. **Type** `NodePort`: 외부에서 웹 애플리케이션에 접근할 수 있도록 설정한다.
- ii. **NodePort:** Kubernetes 노드에서 열리는 포트로, 30,000 ~ 32,767 사이의 값을 사용한다.

7. 환경 변수 설정:

a. MongoDB:

- i. 이미지 문서에서 확인한 환경 변수 이름
(`MONGO_INITDB_ROOT_USERNAME` , `MONGO_INITDB_ROOT_PASSWORD`)을 사용한다.
 - ii. `valueFrom` 을 사용하여 Secret에서 값을 참조한다.
- b. 웹 애플리케이션:
- i. 데이터베이스 연결 정보(URL, 사용자 이름, 비밀번호)를 환경 변수로 전달한다.
 - ii. ConfigMap에서 URL(`db_url`)을 참조하고, Secret에서 사용자 이름(`username`)과 비밀번호(`user_pwd`)를 참조한다.

3.3. Kubernetes 구성 요소 배포 및 확인

1. 구성 요소 생성:

- a. `kubectl apply -f <filename.yaml>` 명령어를 사용하여 ConfigMap, Secret, Deployment, Service를 순차적으로 생성한다.

2. 생성된 구성 요소 확인:

- a. `kubectl get all` : Deployment, Pod, Service 등 클러스터의 모든 구성 요소를 보여준다.
- b. `kubectl get configmap` : ConfigMap 목록을 확인한다.
- c. `kubectl get secret` : Secret 목록을 확인한다.

3. 자세한 정보 확인:

- a. `kubectl describe <component_type> <instance_name>` : 특정 구성 요소의 상세 정보를 확인한다. (예: `kubectl describe service webapp-service`)
- b. `kubectl describe pod <pod_name>` : Pod의 스케줄링 상태, 컨테이너 구성 등 상세 정보를 확인한다.

4. 로그 확인:

- a. `kubectl logs <pod_name>` : Pod 내부 컨테이너의 로그를 확인한다.
- b. `-f` 옵션을 사용하면 로그를 스트리밍할 수 있다.

3.4. 애플리케이션 접근 및 검증

1. 서비스 IP 및 포트 확인:

- a. `kubectl get service` 또는 `kubectl svc` 명령으로 서비스 정보를 확인한다.

- b. 웹 애플리케이션에 접근하기 위한 노드 IP 주소와 노드 포트(30,000-32,767 범위)를 확인한다.

2. 노드 IP 주소 확인:

- a. `minikube ip` 명령어를 사용하거나, `kubectl get node` 명령의 상세 출력에서 노드의 IP 주소를 확인할 수 있다.

3. 애플리케이션 접근:

- a. 브라우저에서 `[노드 IP 주소]:[노드 포트]` 형식으로 웹 애플리케이션에 접속한다.

4. 데이터베이스 연결 검증:

- a. 웹 애플리케이션에서 코드 수정 후 저장하면 변경 사항이 데이터베이스에 반영되는지 확인하여 연결을 검증한다.

3.5. 추가 학습 자료 및 과정 추천

1. 이 강좌를 통해 Kubernetes의 기본 개념과 실제 배포 방법을 익혔다.
2. Kubernetes 전문가가 되고 싶다면, 처음부터 클러스터를 구축하고 관리하는 방법을 배우는 '**완벽한 쿠버네티스 관리자 과정**'을 추천한다.
3. DevOps 엔지니어가 목표라면, Kubernetes를 포함한 필수 개념과 기술을 배우는 '**완벽한 데브옵스 교육 프로그램**'이 적합하다.
4. 자세한 내용은 영상 설명란의 링크를 참조한다.